Name: Eavan Tan **ID: 24006152** **Group_CS_G2**

Assignment 2

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

BFS is built to find the path with the fewest edges instead of the shortest distance. It uses FIFO queue to explore the states to identify how many hops they are from the start. So, the order doesn't directly affect the cost.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

1) frontier = deque([start]) to frontier = [start] (a queue becomes a stack)
2) state = frontier.popleft() to state = frontier.pop() (FIFO becomes LIFO)

3. **UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?**

When at the first time, the goal is generated, it might already have an expensive path. A cheaper path could still be waiting in the frontier. UCS tests the pop time is safe because the popped state has the smallest g in the whole frontier. With non-negative costs, nothing left in the frontier can produce a cheaper path.

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

I should use expanded to differentiate because it is more deterministic. This is because different algorithm works on the same graph and query is possible to expand different the number of states. In the same way if we are using the same algorithm on both machines, both results would just be the same regardless the machine hardware. While runtime's ms depends on CPU speed, background programs, Python version, power mode, and so on.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?

GBFS ignores g(n) which is the cost already travelled. It ranks states only by h(n) which is the heuristic value, "how close to the goal it looks." A* uses g + h.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

Algorithm	**Priority**	**X**	**Y**	**Picks**
UCS	g	8	3	Y
GBFS	h	2	6	X
A*	g + h	10	9	Y

7. Why must *h* be expressed in the same units as the edge costs?

A* computes f = g + h. We can only add two numbers if they measure the same thing and units. For example, adding km to minutes, or km to metres, is wrong.

8. Lab2 computes $h^\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?

h*(n) is the cheapest cost from n to the goal. A Dijkstra run gives costs from one source to all states. If you reverse every edge and start at the goal, one run gives the cost from every airport to the goal. The reversal is needed because flights are directed for example, a route A to B doesn't mean B to A exists or costs the same.

9. Weighted A⋆ with $w = 1.5$ changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

The optimal SIN to EZE route (SIN to JNB to GRU to EZE) runs almost along the straight great-circle line toward the goal. Each hop makes about as much progress toward EZE as it costs, so inflating h never makes a detour look better. Even plain GBFS found this exact route.